

Atividade 03 – Congruência de Zeller

Andrei Formiga

May 14, 2026

Sumário

1. Introdução 3

2. Parte 1: Congruência de Zeller 4

3. Parte 2 - Perguntas 5

1. Introdução

O objetivo desta atividade é criar um programa em assembly x86-64 para calcular a congruência de Zeller, uma fórmula que permite calcular o dia da semana para qualquer data.

2. Parte 1: Congruência de Zeller

A fórmula da congruência de Zeller é:

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + k + \left\lfloor \frac{k}{4} \right\rfloor + \left\lfloor \frac{j}{4} \right\rfloor - 2j \right) \bmod 7 \quad (1)$$

Nesta fórmula, h é um número que vai representar o dia da semana, com 0 representando o sábado, 1 o domingo, 2 a segunda-feira, e por aí vai, até 6 que representa a sexta-feira.

q é o dia do mês; m é um número que representa o mês, com valor entre 3 e 14: 3 é março, 4 é abril, até 12 que é dezembro, depois 13 para janeiro e 14 para fevereiro. k e j juntos representam o ano *ajustado*, com k sendo o ano dentro do século (a unidade e dezena do ano) e j sendo a parte do século (centena e milhar do ano). Por exemplo, para datas a partir de março de 2026 o j é 20 e o k é 26; mas para datas em janeiro e fevereiro (de qualquer ano), o ano é considerado como sendo o ano anterior. Por exemplo, datas em janeiro e fevereiro de 2024 vão ter $j = 20$ e $k = 23$.

Como todos os valores usados são positivos, as divisões com a função piso ($\lfloor \frac{x}{y} \rfloor$) são apenas divisões inteiras, que são calculadas facilmente com a instrução IDIV. A função mod calcula o resto da divisão, portanto na fórmula da congruência toda a quantia dentro dos parênteses é calculada e o resultado final é o resto da divisão dessa quantia por 7.

A parte 1 da atividade consiste em escrever um programa em linguagem *assembly* x86-64 que calcula o valor da congruência de Zeller, considerando o valor de q no registrador R8, m no registrador R9, k em R10 e j em R11. Para testar, coloque valores adequados nesses registradores. Escreva também um pequeno programa em linguagem de alto nível que calcula a congruência, para verificar os resultados obtidos no programa em *assembly*.

Entrega: o arquivo com o programa *assembly* que calcula a congruência de Zeller e o arquivo com o programa em linguagem de alto nível usado para verificar os resultados.

3. Parte 2 - Perguntas

Em breve vamos criar compiladores que geram código *assembly* para expressões aritméticas arbitrárias, e com isso é preciso pensar em quantos registradores são necessários para uma expressão qualquer.

Responda as seguintes questões sobre a tradução de expressões aritméticas para *assembly* x86-64:

1. Qual o menor número de registradores necessários para calcular expressão

$$a_1 + a_2 + \dots + a_n \quad (2)$$

em *assembly*? (Todos os $a_1, a_2, \dots, a_n$ são inteiros constantes.)

2. Qual o menor número de registradores necessários para calcular expressão

$$(a_{11} * a_{12} * \dots * a_{1n}) + \dots + (a_{m1} * a_{m2} * \dots * a_{mn}) \quad (3)$$

em *assembly*? (Todos os a_{ij} são inteiros constantes.)

3. Existe alguma forma de calcular expressões aritméticas constantes (com soma, subtração, multiplicação e divisão) de tamanho arbitrário com um número limitado de registradores, ou o número de registradores pode crescer sem limite? Tente justificar sua resposta.

Entrega: as respostas para as três questões acima.